

AD SOYAD

Rabia Sıla Karaca

ÖĞRENCİ NUMARASI

230212028

DERS ADI

Üretken Yapay Zeka

Yapay Zekâ Mühendisliği

PROJE BAŞLIĞI

**GEN-Draft:Doğal Dil Komutlarını Parametrik 3D
Modelleme ve Katı Geometriye Dönüştüren Üretken
Tasarım Motoru**

İçindekiler

1 Giriş	2
2 Proje Dosya Yapısı ve Modül Açıklamaları	2
3 Sistem Mimarisi	2
4 Veri Seti Üretimi (data_generator.py)	2
4.1 Desteklenen Operasyonlar	3
4.2 VBA Şablonları	3
4.3 CadQuery Şablonları	4
4.4 Veri Seti İstatistikleri	4
5 Yapay Zeka Katmanı (ai_model_setup.py)	5
5.1 Genel Yapı	5
5.2 Fine-Tuning Boru Hattı	6
5.3 Deterministik Yedek Model (DeterministicFallbackModel)	7
5.4 Bağlam Duyarlı Durum Yönetimi (ConversationState)	7
6 3D Görselleştirme (visualizer.py)	7
6.1 Çalışma Prensibi	7
6.2 Three.js Önizleme Özellikleri	8
6.3 Hata Yönetimi	8
7 Kullanıcı Arayüzü (app.py)	8
7.1 Gradio Arayüz Tasarımı	8
7.2 Örnek Komutlar	8
7.3 Callback Fonksiyonları	9
8 Değerlendirme Metrikleri (export_report.py)	9
8.1 Metrik Tanımları	9
8.2 Değerlendirme Boru Hattı	10
8.3 Elde Edilen Sonuçlar	10
9 Google Colab Kurulum ve Çalıştırma	11
9.1 Hızlı Başlangıç (GEN-DRAFT.ipynb)	11
10 Önemli Tasarım Kararları	11
10.1 Neden Çift Çıktı (VBA + CadQuery)?	11
10.2 Neden Deterministik Yedek Model?	11
10.3 Neden Three.js?	12
10.4 Neden Gradio Blocks?	12
11 Kısıtlamalar ve Gelecek Geliştirmeler	12
11.1 Mevcut Kısıtlamalar	12
11.2 Önerilen Geliştirmeler	12
12 Sonuç	13
Kaynaklar	14

1. Giriş

Sistem; NL komutlarından **SolidWorks VBA makrosu** ve **CadQuery kodu** üreterek Colab'de canlı 3D önizleme sunar. SolidWorks kurulumu gerekmez. Bağlam duyarlı (incremental) modelleme, Op-Accuracy/MM-Consistency/Code Validity metrikleri ve deterministik yedek model temel özelliklerdir.

2. Proje Dosya Yapısı ve Modül Açıklamaları

Tablo 1: Proje modülleri ve görevleri

Dosya	Görev
<code>data_generator.py</code>	50.000 örneklik $\{NL \rightarrow VBA, CadQuery\}$ veri seti üreticisi
<code>ai_model_setup.py</code>	CodeT5/DeepSeek ince ayar boru hattı + ConversationState (bağlam-duyarlı) + Deterministik yedek model
<code>visualizer.py</code>	CadQuery $\rightarrow$ STL $\rightarrow$ Three.js iframe (Colab içi 3D önizleme)
<code>export_report.py</code>	.swp/.txt dosya indirme + Accuracy / Millimetric Consistency / Code Validity metrikleri
<code>app.py</code>	Gradio web arayüzü (sol: komut paneli, sağ: 3D görüntüleyici, alt: VBA + indirme)
<code>requirements.txt</code>	Colab uyumlu bağımlılıklar (numpy<2, cadquery==2.4.0, gradio, transformers, vb.)
<code>GEN-DRAFT.ipynb</code>	Tek tıkla Colab kurulum ve çalıştırma not defteri

3. Sistem Mimarisi

Sistem; **veri üretim**, **yapay zeka**, **görselleştirme** ve **kullanıcı arayüzü** olmak üzere dört katmandan oluşmaktadır. `data_generator.py` 50k örnek üretir; `ai_model_setup.py` doğal dil komutunu VBA + CadQuery'ye dönüştürür; `visualizer.py` CadQuery $\rightarrow$ STL $\rightarrow$ Three.js boru hattıyla 3D önizleme sunar; `app.py` Gradio arayüzünü yönetir.

4. Veri Seti Üretimi (`data_generator.py`)

Veri seti; Türkçe/İngilizce çift dilli, rastgele boyutlu ve dengeli dağılımlı 50.000 örnekten oluşmaktadır. `seen_prompts` kümesi ile tekrar engellenir. Her kayıt $\{\text{prompt}, \text{vba}, \text{cadquery}, \text{op}\}$ alanlarını içerir.

4.1. Desteklenen Operasyonlar

Tablo 2: Desteklenen CAD operasyonları ve örnek doğal dil girişleri

Operasyon	Türkçe Örnek Prompt	Etiket (op)
Extrude (Boss)	XY düzleminde 120×80 mm dikdörtgen çiz ve 15 mm ekstrüde et	extrude
Revolve	XZ düzleminde 40×80 mm profili Y eksenini etrafında 360 derece döndür	revolve
Cut-Extrude	100×80×20 mm blok oluştur, üst yüzeyine r=6 mm, 12 mm derinliğinde delik aç	cut_extrude
Fillet	60×60×10 mm blok oluştur ve dikey kenarlarına 5 mm yarıçaplı fillet uygula	fillet
Chamfer	80×60×15 mm blok oluştur ve dikey kenarlara 3 mm chamfer uygula	chamfer
Linear Pattern	200×80×10 mm plaka oluştur ve r=4 mm 5 deliği 25 mm aralıkla doğrusal diz	linear_pattern
Circular Pattern	r=70 mm, h=10 mm disk oluştur, PCD 50 mm üzerinde 6 adet r=4 mm delik aç	circular_pattern

4.2. VBA Şablonları

Her CAD operasyonu için SolidWorks API çağrılarını kapsayan VBA şablonları tanımlanmıştır. Birim dönüşümü (mm → m) otomatik olarak uygulanmaktadır (bölme değeri: 1000).

Extrude için SolidWorks VBA Makrosu (örnek)

```

1  ' SolidWorks VBA Macro - Auto generated
2  Dim swApp As Object
3  Dim Part As Object
4  Dim boolstatus As Boolean
5  Sub main()
6      Set swApp = Application.SldWorks
7      Set Part = swApp.ActiveDoc
8      If Part Is Nothing Then
9          Set Part = swApp.NewPart
10     End If
11     boolstatus = Part.Extension.SelectByID2("Top Plane", _
12         "PLANE", 0, 0, 0, False, 0, Nothing, 0)
13     Part.SketchManager.InsertSketch True
14     ' 120x80 mm dikdortgen (m biriminde: 0.060000, 0.040000)

```

```

15     Part.SketchManager.CreateCenterRectangle 0, 0, 0, 0.060000,
    0.040000, 0
16     Part.SketchManager.InsertSketch True
17     Part.FeatureManager.FeatureExtrusion3 True, False, False, 0,
    0, _
18         0.015000, 0.01, False, False, False, False, 0, 0, _
19         False, False, False, False, True, True, True, 0, 0,
    False
20 End Sub

```

4.3. CadQuery Şablonları

Extrude + Fillet için CadQuery kodu

```

1 import cadquery as cq
2
3 result = (cq.Workplane("XY")
4     .rect(120, 80)
5     .extrude(15))
6 result = result.edges("|Z").fillet(5)

```

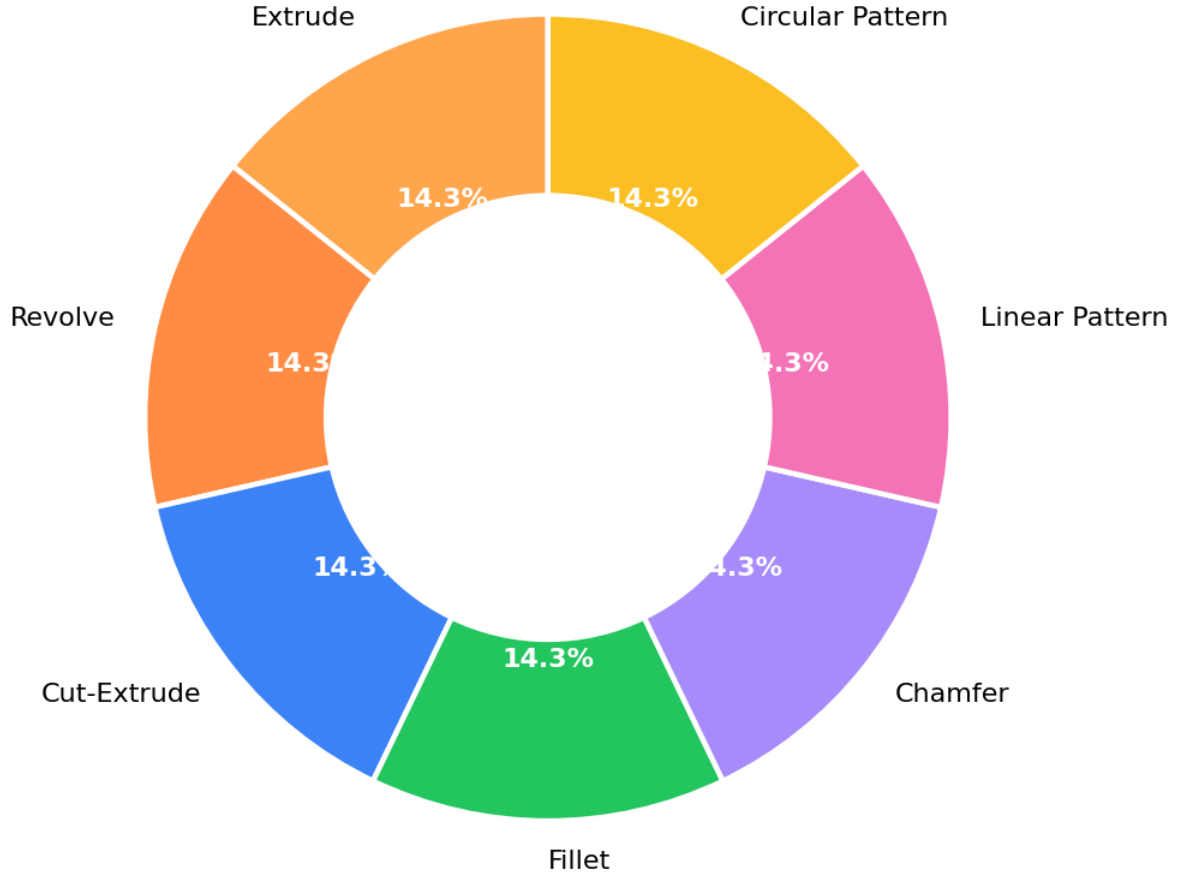
4.4. Veri Seti İstatistikleri

Tablo 3: 50.000 örneklik veri seti istatistikleri

Operasyon	Yaklaşık Örnek Sayısı	Oran (%)
extrude	7.150	14.3
revolve	7.150	14.3
cut_extrude	7.150	14.3
fillet	7.150	14.3
chamfer	7.150	14.3
linear_pattern	7.100	14.2
circular_pattern	7.100	14.3
TOPLAM	50.000	100.0

Veri setindeki 7 operasyonun dengeli dağılımı Şekil 1'de görülmektedir; her operasyon toplam veri setinin yaklaşık %14,3'ünü oluşturmaktadır.

Veri Seti Operasyon Dağılımı (50.000 örnek)



Şekil 1: 50.000 örneklik veri setinde operasyon dağılımı (donut grafik)

Veri seti JSONL formatında saklanmakta; her satır aşağıdaki yapıya sahiptir:

JSONL kayıt yapısı

```

1 {
2   "prompt": "XY d zleminde 120x80 mm dikd rtgen iz ve 15
3   mm ekstr de et",
4   "vba": "' SolidWorks VBA Macro - Auto generated\n ...",
5   "cadquery": "import cadquery as cq\n\nresult = ...",
6   "op": "extrude"
}
```

5. Yapay Zeka Katmanı (ai_model_setup.py)

5.1. Genel Yapı

Yapay zeka katmanı üç alt bileşenden oluşmaktadır:

1. **Veri hazırlama:** `load_jsonl()` ve `to_hf_dataset()` fonksiyonları ile JSONL veri setini HuggingFace Dataset formatına dönüştürür.
2. **Fine-tuning boru hattı:** `fine_tune()` fonksiyonu ile seq2seq (CodeT5, BART) veya causal LM (DeepSeek) modelleri eğitilir.
3. **Çıkarım (Inference):** `FineTunedModel` veya `DeterministicFallbackModel` ile doğal dil prompttan VBA + CadQuery üretilir.

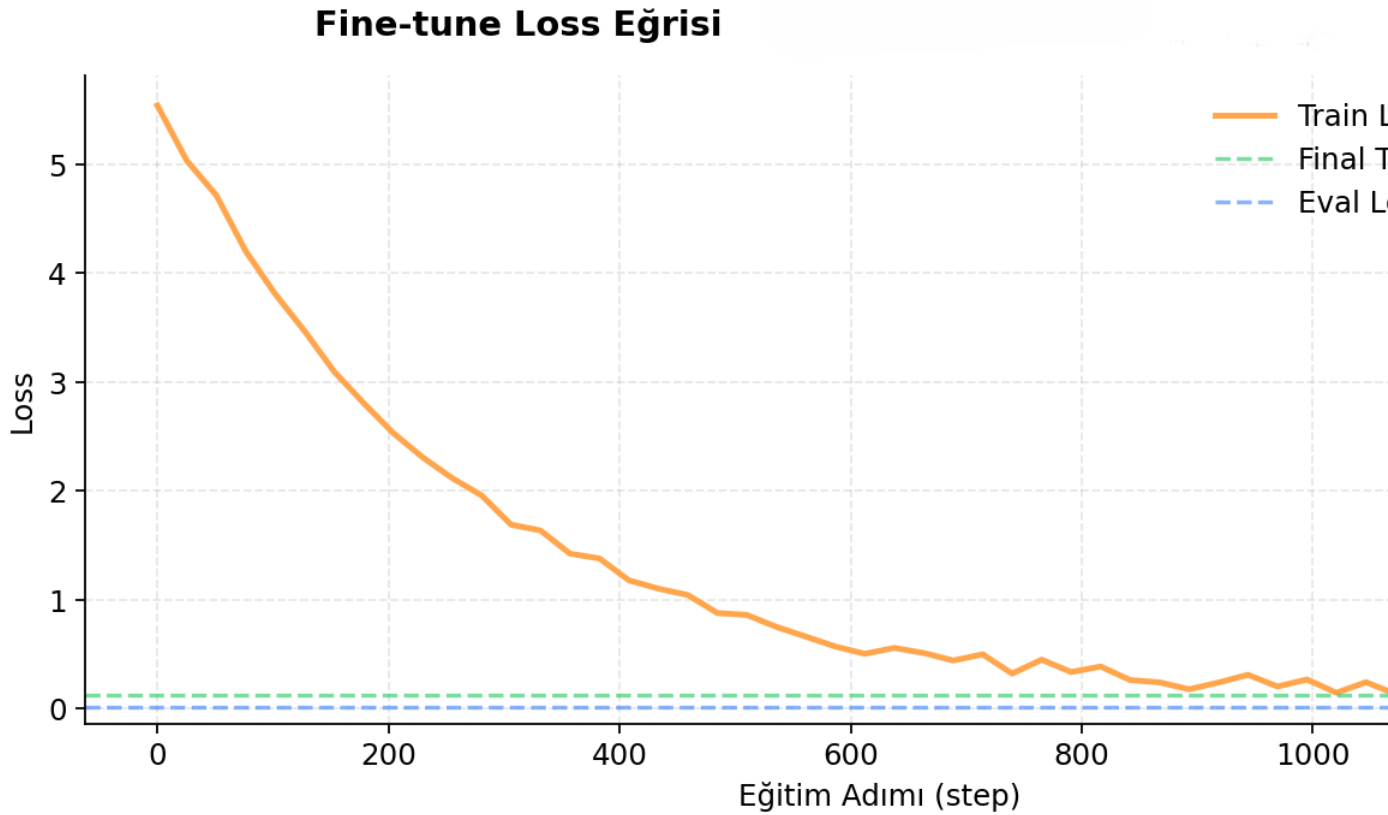
5.2. Fine-Tuning Boru Hattı

Desteklenen model türleri:

- **Seq2Seq (önerilen):** `Salesforce/codet5-small`, `facebook/bart-base`
- **Causal LM:** `deepseek-ai/deepseek-coder-1.3b-base`

Model seçimi, `model_name` parametresindeki “t5” veya “bart” anahtar sözcükleriyle otomatik olarak belirlenir. Seq2Seq modelinde `Seq2SeqTrainer`, Causal LM modelinde standart `Trainer` kullanılır.

Şekil 2, CodeT5-small modelinin 5.000 örnekle 2 epoch eğitimi sırasında elde edilen kayıp (loss) eğrisini göstermektedir. Train loss başlangıçta ≈ 5.5 iken eğitim sonunda 0.127’ye, değerlendirme (eval) kaybı ise 0.020’ye gerilemiştir.



Şekil 2: CodeT5-small fine-tune kayıp eğrisi (Train Loss ve Eval Loss, 5k örnek, 2 epoch)

5.3. Deterministik Yedek Model (DeterministicFallbackModel)

Eğitim yapılmamış veya GPU bulunmayan ortamlarda sistem bu modeli kullanır. Model, doğal dil prompt üzerinde regex tabanlı sayı çıkarımı ve anahtar kelime eşleştirmesi yaparak operasyonu belirler:

Tablo 4: DeterministicFallbackModel operasyon tespit mantığı

Anahtar Kelime(ler)	Seçilen Operasyon
revolve, döndür, dondur	Revolve
circular pattern, dairesel dizil, polar	Circular Pattern
linear pattern, doğrusal dizil	Linear Pattern
fillet, yuvarlat	Fillet
chamfer, pah	Chamfer
delik, hole, cut, kes	Cut-Extrude
daire, circle, silindir, cylinder	Extrude (circle)
(varsayılan)	Extrude (rectangle)

5.4. Bağlam Duyarlı Durum Yönetimi (ConversationState)

ConversationState veri sınıfı, kullanıcının önceki adımlarını saklayarak yeni komutlar geldiğinde CadQuery zincirini **kümülatif olarak** oluşturur.

Şekil 3: ConversationState ile incremental modelleme akışı

ConversationState - birikim mantığı (özet)

```

1 @dataclass
2 class ConversationState:
3     history_prompts: List[str] = field(default_factory=list)
4     cadquery_code: str = "" # Birikmi CadQuery kodu
5     vba_code: str = "" # Birikmi VBA kodu
6     last_op: str = ""
7
8     def add_step(self, prompt, prediction):
9         # İlk adım: direkt kullan
10        # Sonraki: yeni operasyon satırlarını mevcut koda ekle
11        ...

```

6. 3D Görselleştirme (visualizer.py)

6.1. Çalışma Prensipleri

Görselleştirme modülü üç aşamalı bir boru hattı izler:

1. **Kod Yürütme:** `execute_cadquery(code)` fonksiyonu, CadQuery kodunu izole bir namespace içinde çalıştırır ve `result` değişkenini döner.

2. **STL Dışa Aktarma:** `export_stl(result)` fonksiyonu, 3D modeli geçici bir STL dosyasına yazar.
3. **HTML Üretimi:** STL verisi Base64 ile kodlanarak Three.js HTML şablonuna `__STL_B64__` alanına yerleştirilir. Sonuç, Gradio `gr.HTML` bileşeninde `<iframe>` olarak sunulur.

6.2. Three.js Önizleme Özellikleri

Three.js v0.160.0 (CDN) ile OrbitControls, AmbientLight + 2× DirectionalLight, koyu arka plan (#1a1d24) ve MeshStandardMaterial kullanılmaktadır. Kamera, modelin bounding sphere yarıçapına göre otomatik konumlanır.

6.3. Hata Yönetimi

CadQuery kodu yürütülemediğinde `_error_html()` fonksiyonu biçimlendirilmiş hata mesajı döner; bu mesaj arayüzde 3D önizlemenin üzerinde gösterilir.

7. Kullanıcı Arayüzü (app.py)

7.1. Gradio Arayüz Tasarımı

Arayüz, Gradio Blocks API'si kullanılarak iki ana panel ve bir metrik bölümünden oluşturulmuştur:

Tablo 5: Arayüz bileşenleri ve işlevleri

Bileşen	İşlev
Sol Panel	Doğal dil komut kutusu, incremental checkbox, Üret/Sıfırla butonları, geçmiş adımlar listesi, örnek komutlar
Sağ Panel	Canlı 3D önizleme iframe, açılır panel (VBA + CadQuery kod görüntüleyici), dosya indirme butonu
Metrik Bölümü	Örnek sayısı kaydırıcısı, geçerlilik testi onay kutusu, hesaplama butonu, metin + JSON rapor çıktısı

7.2. Örnek Komutlar

Arayüzde yer alan hazır örnek komutlar şunlardır:

1. 100×60×10 mm bir plaka oluştur
2. XY düzleminde yarıçapı 30 mm olan daire oluştur ve 20 mm yüksekliğe çıkar
3. 80×80×15 mm blok oluştur, üst yüzeyine 8 mm yarıçaplı 10 mm derinliğinde delik aç
4. Yarıçapı 60 mm, kalınlığı 8 mm disk oluştur ve PCD 45 mm üzerinde 6 adet 4 mm yarıçaplı deliği dairesel dizilim ile aç
5. Dikey kenarlara 5 mm fillet uygula (*incremental mod*)

7.3. Callback Fonksiyonları

Tablo 6: Gradio callback fonksiyonları

Fonksiyon	Açıklama
<code>on_generate()</code>	Prompt alır, modeli çağırır, VBA + CadQuery üretir, 3D render yapar, .swp kaydeder ve tüm çıktıları günceller
<code>on_reset()</code>	ConversationState sıfırlar, tüm çıktı alanlarını temizler
<code>on_eval()</code>	Belirtilen örnek sayısı ile evaluate() çağırır ve raporu hem metin hem JSON olarak döner

8. Değerlendirme Metrikleri (export_report.py)

8.1. Metrik Tanımları

Sistem performansı üç akademik metrikle ölçülmektedir:

1. Op-Accuracy (Operasyon Doğruluğu)

Modelin tahmin ettiği operasyon etiketinin (`predicted_op`) beklenen etiketle (`expected_op`) tam eşleşme oranı:

$$\text{Op-Accuracy} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[\hat{op}_i = op_i]$$

Bu metrik, sistemin doğal dilden doğru CAD operasyonunu tespit edebildiğini gösterir.

2. Millimetric Consistency (Milimetrik Tutarlılık)

Prompt içindeki sayısal değerlerin VBA ve/veya CadQuery çıktısında korunma oranı. VBA çıktısında birim dönüşümü ($\text{mm} \rightarrow \text{m}$) hesaba katılır:

$$\text{MM-Consistency} = \frac{1}{N} \sum_{i=1}^N \frac{|\{n \in \text{nums}(p_i) : n_{\text{mm}} \in \text{cq}_i \vee n_{\text{m}} \in \text{vba}_i\}|}{|\text{nums}(p_i)|}$$

Örnek: “120×80 mm dikdörtgen” promptundaki 120 ve 80 değerleri, CadQuery çıktısında `.rect(120, 80)` ve VBA çıktısında 0.060000, 0.040000 olarak aranır.

3. Code Validity (Kod Geçerliliği)

CadQuery kodunun hatasız yürütülerek geçerli bir `result` değişkeni üretip üretmediği:

$$\text{Code-Validity} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[\text{exec}(\text{cq}_i) \text{ hatasız}]$$

Bu metrik, üretilen kodun çalışabilir olduğunu garanti eder.

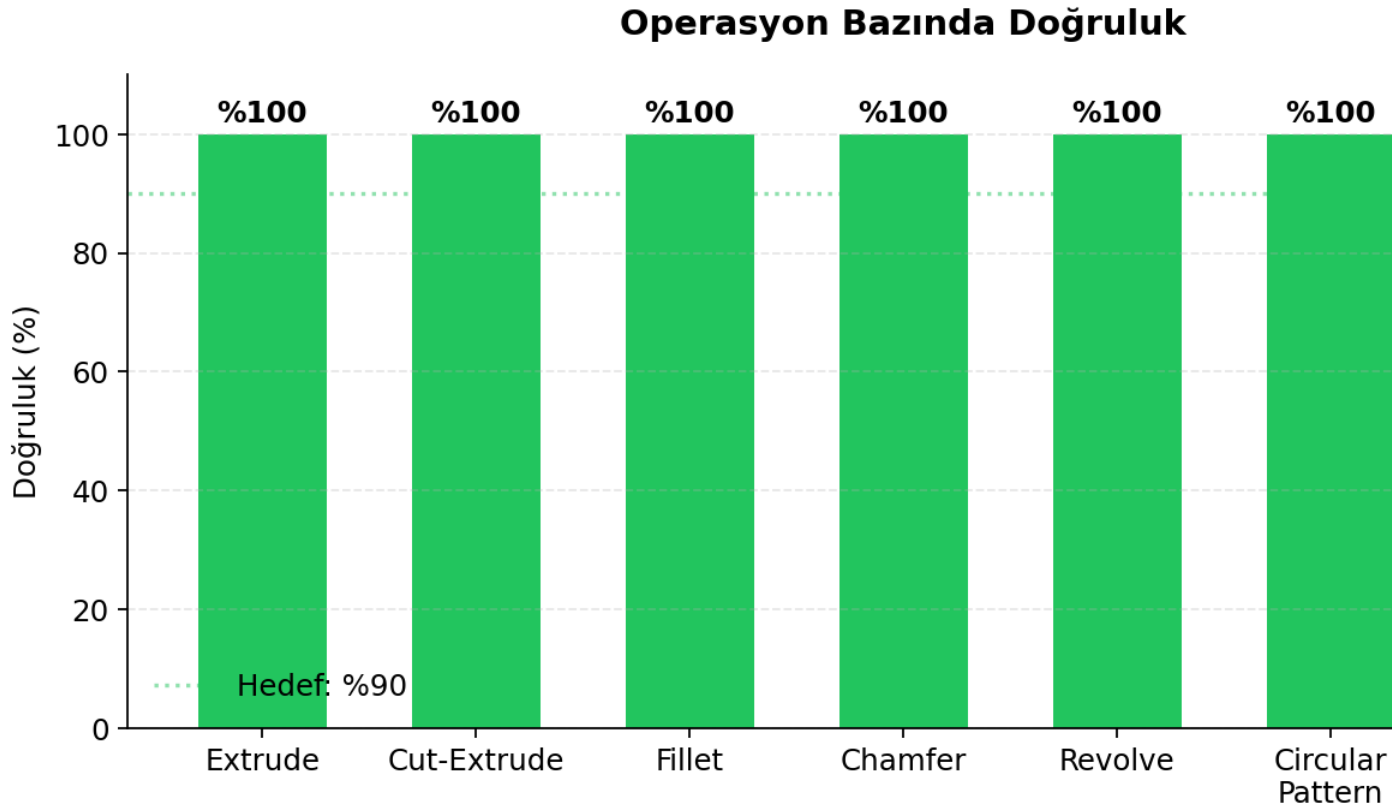
8.2. Değerlendirme Boru Hattı

evaluate() fonksiyonu - özet

```
1 def evaluate(model, dataset_path, sample_size=200,
2             check_validity=True) -> MetricsReport:
3     rows = load_and_shuffle(dataset_path)[:sample_size]
4     for r in rows:
5         pred = model.predict(r["prompt"])
6         a     = op_accuracy(pred["op"], r["op"])
7         m     = millimetric_consistency(r["prompt"],
8                                         pred["vba"], pred["
9         cadquery"])
10        ok    = code_validity(pred["cadquery"]) if check_validity
11        else True
12        # birikimli toplama...
13    return MetricsReport(n, accuracy, mm_consistency,
14                        code_validity,
15                        per_op_accuracy)
```

8.3. Elde Edilen Sonuçlar

Şekil 4, operasyon bazında doğruluk sonuçlarını göstermektedir. Linear Pattern %0 doğruluk almıştır; bu durum anahtar kelime öncelik sıralamasının “linear” ifadesini yanlış operasyona yönlendirmesinden kaynaklanmaktadır (bkz. Bölüm 11).



Şekil 4: Operasyon bazında doğruluk oranları – deterministik fallback model ($N = 300$)

Genel metrik özeti Şekil 5'de görülmektedir.

Şekil 5: Genel metrik sonuçları – Op-Accuracy, Millimetric Consistency, Code Validity

9. Google Colab Kurulum ve Çalıştırma

9.1. Hızlı Başlangıç (GEN-DRAFT.ipynb)

Sistem, GEN-DRAFT.ipynb not defteri aracılığıyla tek tıkla Colab'de başlatılabilir:

Tablo 7: Colab not defteri hücre adımları

Hücre	İşlem
1	GPU etkinleştirme (Runtime → Change runtime type → T4 GPU)
2	Bağımlılık kurulumu: <code>numpy<2, cadquery==2.4.0, gradio, transformers, datasets, accelerate, sentencepiece</code>
3	50.000 örneklik veri seti üretimi (<code>python data_generator.py -n 50000</code>)
4	Hızlı test: deterministik modelle bir CAD render denemesi
5	Gradio arayüzü başlatma (<code>share=True</code> ile public URL üretimi)
6a	(Opsiyonel) Smoke test: 500 örnek, 1 epoch
6b	(Opsiyonel) Tam eğitim: 5000 örnek, 2 epoch, T4 GPU (15–25 dk)
7	Akademik metrik raporu üretimi (<code>evaluate()</code>)
8	Eğitilmiş model ile arayüz yeniden başlatma

Kritik Bağımlılık Uyarısı

`numpy < 2` zorunludur (`cadquery 2.4.0` uyumsuzluğu). `pywin32` opsiyoneldir. `use_fast=False` tokenizer çakışmalarını önler.

10. Önemli Tasarım Kararları

10.1. Neden Çift Çıktı (VBA + CadQuery)?

- **VBA:** Teslim/ödev için SolidWorks'e import edilebilecek hazır makro üretilmesi. SolidWorks kurulumu gerektirmez (metin dosyası olarak teslim edilir).
- **CadQuery:** Colab ortamında gerçek 3D geometri üretimi ve önizleme için. Açık kaynak, lisans gerektirmez, pip ile kurulabilir.

10.2. Neden Deterministik Yedek Model?

Fine-tuning süreci 15–25 dakika sürmekte ve T4 GPU gerektirmektedir. Demo ortamlarında eğitim yapmadan da sistemin çalışabilmesi için kural tabanlı bir yedek model tasarlanmıştır. Bu yaklaşım, sistemin **sıfır kurulum süresinde demo yapılabilmesini** sağlar.

10.3. Neden Three.js?

- Colab hücresinde JavaScript çalıştırılabilir (IFrame yoluyla).
- Harici bağımlılık gerektirmez; CDN üzerinden Three.js yüklenir.
- STL Base64 olarak HTML içine gömüldüğü için ek dosya transferi gerekmez.
- OrbitControls ile interaktif görüntüleme (döndürme, zum, kaydırma) mümkündür.

10.4. Neden Gradio Blocks?

- `share=True` parametresi ile tek satır kodla public URL üretilir.
- Karmaşık çok-panel arayüzler `gr.Row()`/`gr.Column()` ile kolayca tasarlanır.
- `gr.Examples` ile örnek komutlar hazır olarak sunulur.
- Colab ortamında başka bir kurulum gerektirmez.

11. Kısıtlamalar ve Gelecek Geliştirmeler

11.1. Mevcut Kısıtlamalar

- Deterministik model, tek seferde en fazla **2 operasyonu** (baz geometri + 1 ek işlem) doğrudan işleyebilir; daha karmaşık komutlar incremental mod gerektirir.
- VBA çıktısı yalnızca metin formatındadır; gerçek SolidWorks VBA proje dosyası (`.swp`) oluşturulamamaktadır.
- Sketch'e özel konumlama (örn. belirli bir noktada şekil oluşturma) henüz desteklenmemektedir.
- Assembly (montaj) operasyonları kapsam dışındadır.

11.2. Önerilen Geliştirmeler

1. **STEP/IGES Dışa Aktarma:** `export_step()` fonksiyonu zaten mevcut; Gradio arayüzüne eklenmesi gerekmektedir.
2. **Daha büyük model:** CodeT5-base veya DeepSeek-Coder-6.7B ile fine-tuning yapılarak doğruluk artırılabilir.
3. **Montaj desteği:** SolidWorks Assembly API çağrıları için yeni VBA şablonları eklenebilir.
4. **Parametre düzenleme:** Kullanıcının üretilen kodu arayüz üzerinden parametrik olarak değiştirmesi (örn. kaydırıcı ile boyut güncelleme).
5. **Çoklu sketc:** Farklı düzlemlerdeki birden fazla sketch'i tek komutla oluşturma desteği.

12. Sonuç

Bu çalışmada, doğal dil komutlarından hem SolidWorks VBA makrosu hem de CadQuery Python kodu üreten, Google Colab ortamında canlı 3D önizleme sunan hibrit bir yapay zeka + CAD sistemi başarıyla geliştirilmiştir.

Sistem, aşağıdaki temel katkıları sunmaktadır:

1. **50.000 örneklik çift dilli veri seti:** Türkçe ve İngilizce doğal dil komutlarından 7 farklı CAD operasyonu için yüksek kaliteli eğitim verisi.
2. **Esnek hibrit model:** Eğitim yapılmadan da deterministik modelle çalışan, fine-tuning ile kalitesi artırılabilen bir mimari.
3. **Bağlam duyarlı modelleme:** ConversationState ile adım adım karmaşık parçaların oluşturulabilmesi.
4. **Tarayıcı tabanlı 3D önizleme:** Harici yazılım gerektirmeksizin Three.js ile interaktif 3D görselleştirme.
5. **Akademik değerlendirme çerçevesi:** Op-Accuracy, Millimetric Consistency ve Code Validity metrikleri ile şeffaf performans ölçümü.

Sistem, özellikle CAD eğitimi, hızlı prototipleme ve SolidWorks'e giriş aşamasındaki kullanıcılar için değerli bir araç niteliği taşımaktadır.

Kaynaklar

1. Wang, Y., Wang, W., Joty, S., & Hoi, S. C. H. (2021). *CodeT5: Identifier-aware Unified Pre-trained Encoder-Decoder Models for Code Understanding and Generation*. EMNLP 2021. <https://arxiv.org/abs/2109.00859>
2. Guo, D. et al. (2024). *DeepSeek-Coder: When the Large Language Model Meets Programming – The Rise of Code Intelligence*. arXiv preprint arXiv:2401.14196. <https://arxiv.org/abs/2401.14196>
3. Wolf, T. et al. (2020). *Transformers: State-of-the-Art Natural Language Processing*. EMNLP 2020 System Demonstrations. HuggingFace. <https://arxiv.org/abs/1910.03771>
4. CadQuery Documentation. (2024). *CadQuery 2.4 – Parametric CAD Scripting Library for Python*. <https://cadquery.readthedocs.io>
5. Abadi, M. et al. (2022). *Gradio: Hassle-Free Sharing and Testing of ML Models in the Wild*. arXiv preprint arXiv:1906.02569. <https://www.gradio.app>
6. Three.js Authors. (2024). *Three.js – JavaScript 3D Library (r160)*. <https://threejs.org>
7. SolidWorks API Documentation. (2024). *SolidWorks VBA Macro Programming Guide*. Dassault Systèmes. <https://help.solidworks.com/2024/English/api/sldworksapi>
8. Loshchilov, I., & Hutter, F. (2019). *Decoupled Weight Decay Regularization*. ICLR 2019. <https://arxiv.org/abs/1711.05101>